Question 2: Manage Docker containers and images: list, start, stop, remove containers, tag and push images to a registry, and use Docker Compose for multi-container setups.

Container lifecycle commands were demonstrated on the lab2-app container, the image was tagged and pushed to Docker Hub, and a multi-container setup (app + redis) was run with Docker Compose.

Container lifecycle

```
$ docker ps --filter name=lab2-app
$ docker stop lab2-app && docker ps -a --filter name=lab2-app
$ docker start lab2-app && docker restart lab2-app
$ curl http://localhost:5000/health
{"status":"UP","version":"2.0.0"}
```

Tag and push to Docker Hub

```
$ docker tag devops-lab2-app:2.0.0 abhinaaav/devops-lab2-app:2.0.0
$ echo <token> | docker login -u abhinaaav --password-stdin
Login Succeeded
$ docker push abhinaaav/devops-lab2-app:2.0.0
2.0.0: digest: sha256:66c210e3f9344d4f0a6cdb4aed54e3e1430af368a951393c4ca0197b0679e1aa size: 856
```

```
abhinav@LAPTOP-BSV7PCJK: ~/devops-lab1-ci - terminal
### Container lifecycle management
abhinav@LAPTOP-BSV7PCJK:~$ docker ps   (running containers)
NAMES      IMAGE                                STATUS          PORTS
lab2-app   devops-lab2-app:2.0.0                Up 8 minutes    0.0.0.0:5000->5000/tcp, [::]:5000->5000/t
minikube   gcr.io/k8s-minikube/kicbase:v0.0.51   Up 13 minutes   127.0.0.1:44620->22/tcp, 127.0.0.1:44621-
abhinav@LAPTOP-BSV7PCJK:~$ docker stop lab2-app
lab2-app
abhinav@LAPTOP-BSV7PCJK:~$ docker ps -a --filter name=lab2-app   (stopped container still exists)
NAMES      STATUS
lab2-app   Exited (137) Less than a second ago
abhinav@LAPTOP-BSV7PCJK:~$ docker start lab2-app
lab2-app
abhinav@LAPTOP-BSV7PCJK:~$ docker restart lab2-app
lab2-app
abhinav@LAPTOP-BSV7PCJK:~$ curl http://localhost:5000/health   (container back up)
### Image management
abhinav@LAPTOP-BSV7PCJK:~$ docker images devops-lab2-app
IMAGE                  ID             DISK USAGE   CONTENT SIZE   EXTRA
devops-lab2-app:2.0.0   66c210e3f934        204MB         49.6MB   U
abhinav@LAPTOP-BSV7PCJK:~$ docker tag devops-lab2-app:2.0.0 abhinaaav/devops-lab2-app:2.0.0
IMAGE                           ID             DISK USAGE   CONTENT SIZE   EXTRA
abhinaaav/devops-lab2-app:2.0.0   66c210e3f934        204MB         49.6MB   U
abhinav@LAPTOP-BSV7PCJK:~$ echo <token> | docker login -u abhinaaav --password-stdin
Login Succeeded
abhinav@LAPTOP-BSV7PCJK:~$ docker push abhinaaav/devops-lab2-app:2.0.0
The push refers to repository [docker.io/abhinaaav/devops-lab2-app]
6fc242d221a6: Waiting
3eb771151506: Waiting
93a7b47bf0c0: Waiting
44136fa355b3: Waiting
93a7b47bf0c0: Waiting
44136fa355b3: Waiting
6fc242d221a6: Waiting
3eb771151506: Waiting
3eb771151506: Waiting
93a7b47bf0c0: Waiting
44136fa355b3: Waiting
6fc242d221a6: Waiting
```

Fig: Container lifecycle commands

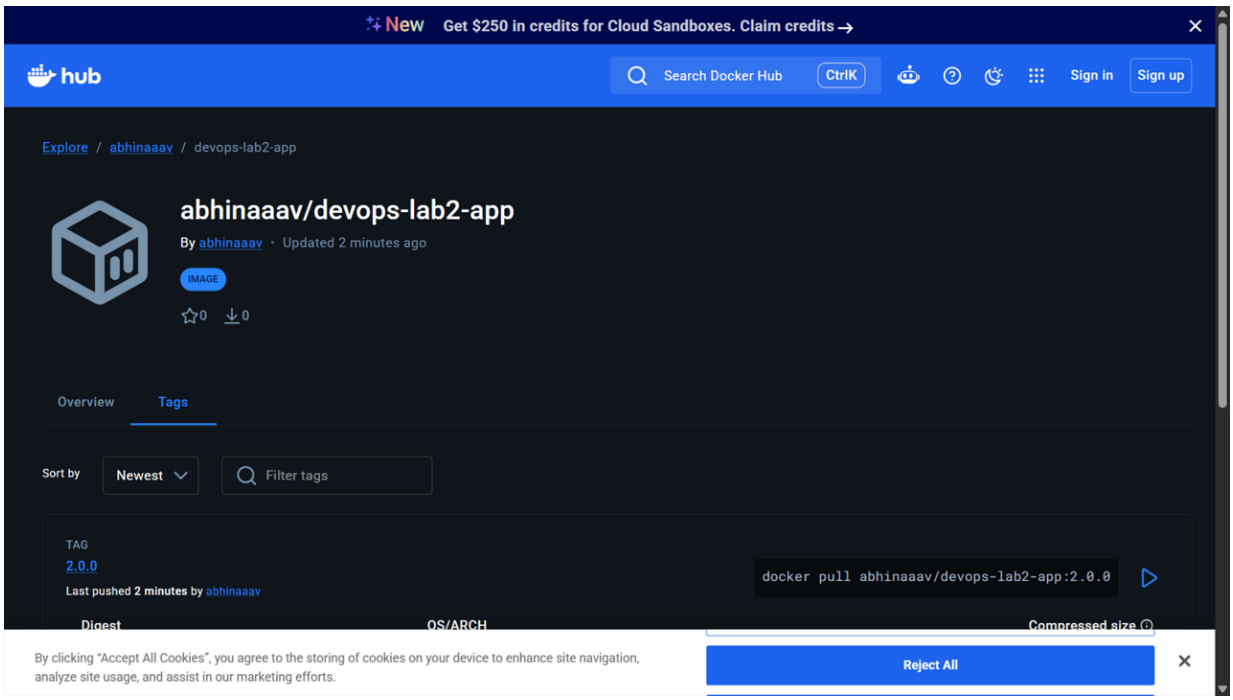


Fig: Docker Hub page - abhinaaav/devops-lab2-app tag 2.0.0 pushed

Docker Compose (web + redis)

```
$ docker compose -f docker/docker-compose.yml up -d
 Container lab2-redis Started
 Container docker-web-1 Started

$ docker compose -f docker/docker-compose.yml ps
docker-web-1  devops-lab2-app:2.0.0  Up  0.0.0.0:5000->5000/tcp
lab2-redis    redis:7-alpine         Up  0.0.0.0:6379->6379/tcp

$ docker compose ... exec web python -c "import socket; print(socket.gethostbyname('redis'))"
redis IP: 172.19.0.2

$ docker compose -f docker/docker-compose.yml down
```

```
abhinav@LAPTOP-BSV7PCJK: ~/devops-lab1-ci  —  terminal
### Docker Compose: multi-container setup (web + redis)
(note: the standalone container from Q1 was stopped first to free port 5000)
abhinav@LAPTOP-BSV7PCJK:~$ docker stop lab2-app && docker rm lab2-app
lab2-app
lab2-app
abhinav@LAPTOP-BSV7PCJK:~$ docker compose -f docker/docker-compose.yml up -d
 Network docker_lab2net Creating
 Network docker_lab2net Creating
 Network docker_lab2net Created
 Network docker_lab2net Created
 Container lab2-redis Creating
 Container lab2-redis Created
 Container docker-web-1 Creating
 Container docker-web-1 Created
 Container lab2-redis Starting
 Container lab2-redis Started
 Container docker-web-1 Starting
 Container docker-web-1 Started
abhinav@LAPTOP-BSV7PCJK:~$ docker compose -f docker/docker-compose.yml ps
NAME           IMAGE                  COMMAND                  SERVICE   CREATED         STATUS
docker-web-1   devops-lab2-app:2.0.0  "python app.py"          web       2 seconds ago   Up Less than a s
lab2-redis     redis:7-alpine         "docker-entrypoint.s…"   redis     2 seconds ago   Up 2 seconds
abhinav@LAPTOP-BSV7PCJK:~$ docker compose -f docker/docker-compose.yml logs web --tail 4
### compose containers resolve each other by service name (own network lab2net)
abhinav@LAPTOP-BSV7PCJK:~$ docker compose ... exec web python -c "import socket; print(\"redis IP:\", sock
redis IP: 172.19.0.2
abhinav@LAPTOP-BSV7PCJK:~$ curl http://localhost:5000/health   (web served by compose)
{"status":"UP","version":"2.0.0"}
abhinav@LAPTOP-BSV7PCJK:~$ docker compose -f docker/docker-compose.yml down
 Container docker-web-1 Stopping
 Container docker-web-1 Stopped
 Container docker-web-1 Removing
 Container docker-web-1 Removed
 Container lab2-redis Stopping
 Container lab2-redis Stopped
 Container lab2-redis Removing
 Container lab2-redis Removed
 Network docker_lab2net Removing
```

Fig: compose up, ps and service-name resolution

```
abhinav@LAPTOP-BSV7PCJK: ~/devops-lab1-ci  —  terminal
 Network docker_lab2net Removed
abhinav@LAPTOP-BSV7PCJK:~$ docker compose -f docker/docker-compose.yml ps -a   (after down: nothing left)
NAME      IMAGE     COMMAND   SERVICE   CREATED   STATUS    PORTS
```

Fig: compose health check and clean down

Observations

- stop keeps the container (Exited), start resumes it, rm removes it - ps -a shows both states.
- tag adds another name to the same image ID; the pushed digest matched the local image.
- Compose creates its own network (docker_lab2net) where services resolve by name.
- The compose web service failed while the standalone container held port 5000 - port conflicts must be freed before `compose up`.

Conclusion

All experiments were performed on Ubuntu 24.04 LTS with Docker, Docker Compose, Minikube and kubectl. The evidence (command logs, screenshots and commit history) is committed in the GitHub repository github.com/theabhinav0231/devops-lab-2 under the docs/ directory.